ADR_010 識別碼採用 UUIDv7

> 狀態：已接受
> 日期：2026-10-14

背景

每張資料表都需要主鍵。識別碼的產生方式影響：

- 寫入效能：B+ 樹索引在不同鍵值分佈下的行為差異很大
- 應用程式能否在寫入資料庫之前就決定編號（冪等、事件內容、跨服務參照都需要）
- 日後拆服務、自動擴縮時是否會衝突
- 編號是否會洩漏營運資訊或被猜出

考慮過的選項

選項	大小	B+ 樹寫入行為	主要缺點
資料庫遞增整數	8 位元組	良好，總是寫在尾端	要寫入資料庫才拿得到編號；連續編號可推算訂單量、容易被猜出；拆服務或分庫時容易衝突
隨機 UUID（v4）	16 位元組	差，插入位置隨機，造成頁分裂、記憶體命中率下降、交易日誌變大	資料量大時寫入效能明顯下降
雪花演算法	8 位元組	良好，依時間遞增	每台機器需要不重複的機器編號，自動擴縮時需要額外的分配機制；需處理時鐘倒退；超過 JavaScript 的整數精度，傳給前端時須轉成字串；沒有統一標準
UUIDv7	16 位元組	良好，前段為時間，依時間遞增	比 8 位元組的做法索引大約大四成；從編號可看出建立時間

決定

- 所有資料表的主鍵採用 UUIDv7，由應用程式產生
- 結帳單與子訂單另設給人閱讀的訂單編號，供會員、賣家與客服溝通使用；系統內部的關聯一律使用主鍵
- 在併發正確性階段做對照實驗：分別以遞增整數、隨機 UUID、UUIDv7 灌入 1,000 萬筆資料，比較寫入速度與索引大小，結果補充於新的 ADR

後果

- 好處：不需要分配機器編號，也不必處理時鐘倒退，對雲端自動擴縮友善；寫入前即可決定編號；編號無法被猜出，也不洩漏營運量
- 代價：索引比 8 位元組的做法大；訂單需要另外維護一組給人看的編號
- 重新檢討的條件：資料量成長到索引大小明顯影響效能時，評估改用 8 位元組的做法